



Institución
Universitaria
Reacreditada en Alta Calidad

ESTRUCTURA DE DATOS Y »»» LABORATORIO «««

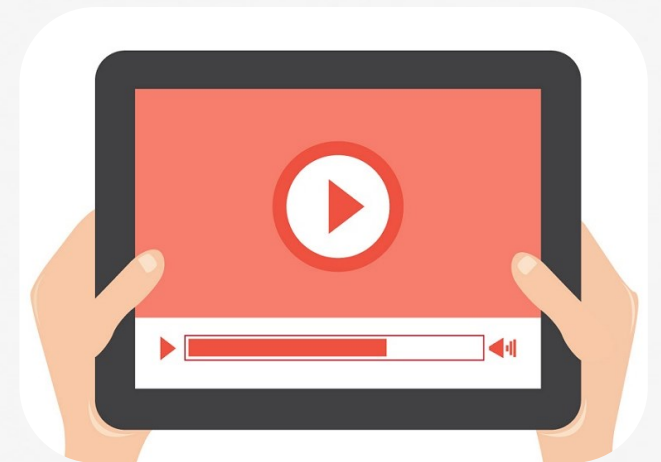
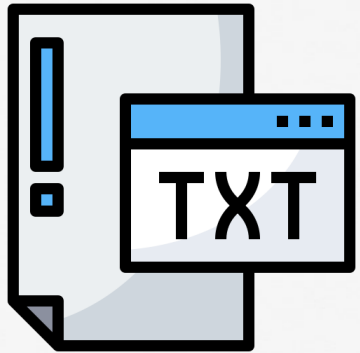
580506004-8

William Giraldo Escobar
Facultad de Ingenierías
Departamento de Sistemas de Información



ARCHIVOS

Un archivo es una unidad de almacenamiento de datos que contiene información digital en formato electrónico o binario.





Institución
Universitaria
Reacreditada en Alta Calidad

ARCHIVOS

Estos datos se organizan de manera estructurada y se almacenan en una ubicación específica en un dispositivo de almacenamiento, como un **disco duro**, una memoria **USB**, una **tarjeta de memoria**, un **servidor en línea** o cualquier otro medio de **almacenamiento digital**.

ARCHIVOS

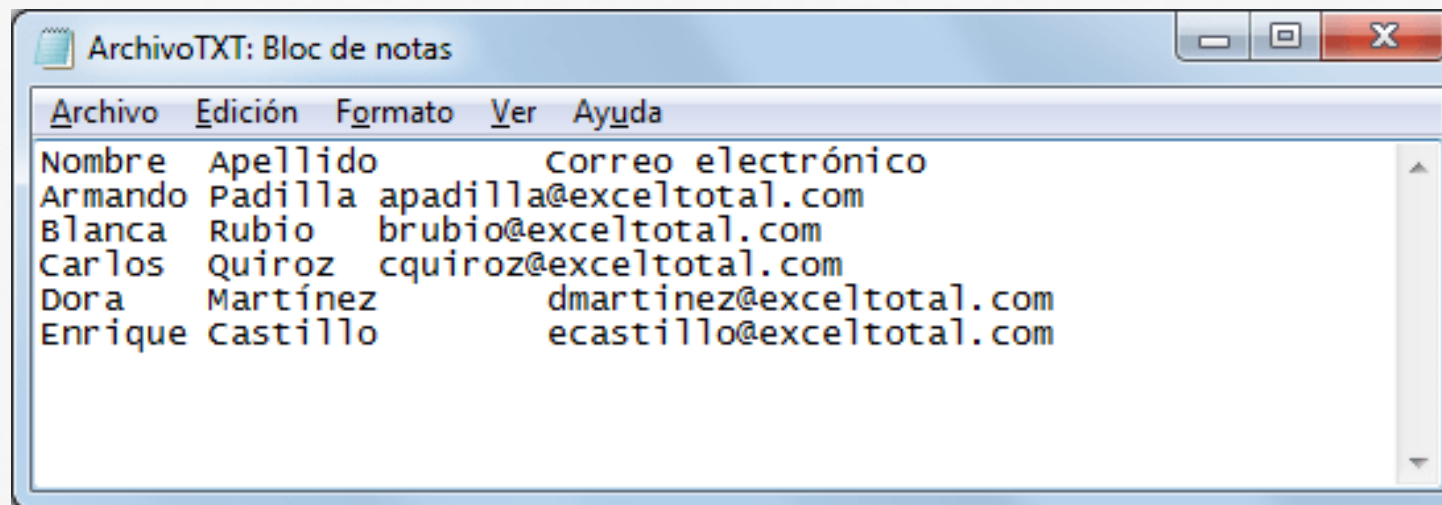
Cada archivo tiene un nombre único y una extensión que indica su tipo o formato

- **.txt** -> Formato de texto
- **.jpg** o **.png** -> Imágenes
- **.docx** -> Documento de Word
- **.py** -> Programa de Python

ARCHIVOS DE TEXTO

Los archivos de texto son uno de los tipos más simples y comunes de archivos utilizados en informática.

Los archivos de texto contienen caracteres alfabéticos y numéricos legibles por humanos.



ARCHIVOS DE TEXTO

Los archivos de texto suelen tener extensiones de archivo como:

- **.txt** -> (Los más comunes y fáciles de usar)
- **.csv** -> (para archivos CSV - Comma-Separated Values)
- **.html** -> (para páginas web)
- **.xml** -> (para documentos estructurados)
- **.json** -> (para datos en formato JSON), entre otros.

ARCHIVOS DE TEXTO

Podemos crear, editar y ver archivos de texto utilizando programas de edición de texto simples, como el **Bloc de notas** en Windows.

También se pueden utilizar aplicaciones más avanzadas, como **Microsoft Word** o editores de código como **Visual Studio Code**, dependiendo del propósito del archivo.

En esta clase aprenderemos cómo operar y manipular archivos de texto desde **Python**.

ARCHIVOS DE TEXTO – GOOGLE COLAB

ACCEDER A ARCHIVOS DE TEXTO EXISTENTES

Si tenemos un archivo de texto almacenado en nuestro ordenador, Podemos acceder a este utilizando los siguientes comandos:

```
from google.colab import files  
  
uploaded_files = files.upload()
```

ARCHIVOS DE TEXTO – GOOGLE COLAB

ACCEDER A ARCHIVOS DE TEXTO EXISTENTES

```
from google.colab import files
```

-> librería de colab para trabajar con archivos

```
uploaded_files = files.upload()
```

-> comando para acceder a los archivos del ordenador

Apertura de archivos – Modo lectura

Puedes abrir un archivo de texto en modo de lectura utilizando la función *open()*

Para abrir un archivo, debes proporcionar la *ruta o nombre del archivo* y el modo en el que deseas abrirlo (lectura, escritura, etc.).

El modo de lectura se especifica utilizando *'r'* (read).

Apertura de archivos – Modo lectura

En la variable ***archivo*** se almacena un objeto de la clase texto.

```
archivo = open('nombre_del_archivo.txt', 'r')
```

Cierre de archivos

Es importante cerrar un archivo después de haberlo utilizado. Puedes cerrarlo utilizando el método ***close()***

Cerrar un archivo es importante porque ***libera los recursos*** del sistema operativo utilizados por el archivo y ***garantiza que todos los cambios*** que hayas realizado se ***guarden correctamente***.



Cierre de archivos

Si no cierras un archivo después de haberlo abierto, podrías enfrentar ***problemas de recursos*** y, en algunos casos, podrías no poder acceder al archivo desde otros procesos hasta que se cierre.

Cierre de archivos

Para cerrar un archivo llamamos al método `.close()` de la clase texto.

Lo hacemos poniendo el nombre de la variable donde almacenamos el objeto texto.

```
archivo.close()
```

Cierre de archivos

NOTA: recuerda **SIEMPRE** cerrar un archivo de texto después de utilizarlo.

```
archivo.close()
```



Lectura de archivos

Para leer el contenido de un archivo de texto, puedes utilizar diversos métodos, como *read()*, *readline()*, o *readlines()*.

Veremos ejemplos de cada uno.

Lectura de archivos – read()

read(): Lee **todo** el contenido del archivo y lo almacena como una **cadena de texto**.

```
contenido = archivo.read()  
print(contenido)  
archivo.close()
```

Lectura de archivos – `readline()`

`readline()`: Lee *una línea del archivo* cada vez que se llama. Puedes usar un bucle para leer todas las líneas *una a una*.

```
linea = archivo.readline()
while linea:
    print(linea)
    linea = archivo.readline()
archivo.close()
```

Lectura de archivos – readline()

```
linea = archivo.readline()
while linea:
    print(linea)
    linea = archivo.readline()
archivo.close()
```

Lectura de archivos – `readline()`

- **`linea = archivo.readline():`** Esta línea de código inicializa la variable `linea` con el contenido de la **primera línea** del archivo.
- **`while linea:`** se ejecuta mientras la variable `linea` tenga un valor (es decir, mientras no sea una cadena vacía o nula).
- **`linea = archivo.readline():`** Después de imprimir una línea, se llama nuevamente a `readline()` para **leer la siguiente línea** del archivo y actualizar el valor de `linea` con esa nueva línea.

Lectura de archivos – `readlines()`

`readlines()`: Lee *todas las líneas* del archivo y las almacena en una *lista*.

```
lineas = archivo.readlines()
for linea in lineas:
    print(linea)

archivo.close()
```



ACTIVIDAD

Lea el archivo nombrado read.txt usando las funciones *.read()*, *.readline()* y *.readlines()* vistos anteriormente.

¿Cuál es la diferencia?

Lectura de archivos – ciclo for

Cuando utilizas un for *directamente* sobre el archivo, Python itera automáticamente sobre las líneas del archivo una por una.

Esto es equivalente al enfoque anterior, pero *es más eficiente* en términos de memoria, ya que *no almacena* todas las líneas en una lista.

Lectura de archivos – With open as y for

```
with open('nombre_del_archivo.txt', 'r') as archivo:  
    for linea in archivo:  
        print(linea)
```

with open('nombre_del_archivo.txt', 'r') as archivo:

Este es un contexto de manejo de archivos. Abre el archivo llamado 'nombre_del_archivo.txt' en modo de lectura ('r') y lo asigna a la variable archivo. El uso de with asegura que el archivo se cierre automáticamente cuando se termine de usar.

for linea in archivo:

En este caso, en lugar de usar readlines(), estamos utilizando un for directamente sobre el archivo archivo. Python automáticamente itera sobre las líneas del archivo, una por una.

Lectura de archivos – With open as y .readlines()

Lee línea por línea y cierra el archivo de forma automática

```
with open('nombre_del_archivo.txt', 'r') as archivo:  
    lineas = archivo.readlines()  
    for linea in lineas:  
        print(linea)
```



Modificar archivos de texto

NOTA: Al modificar un archivo de texto, ten en cuenta que puedes *sobrescribir* su contenido existente si no tienes cuidado. Asegúrate de hacer una *copia de seguridad* del archivo original antes de realizar cambios importantes.

Modificar archivos de texto

Para modificar un archivo de texto primero debes abrirlo en modo escritura (write mode)

Puedes abrir un archivo en modo escritura utilizando la función open con el argumento "w".

Esto *eliminará el contenido existente* del archivo y lo *reemplazará* por el nuevo contenido que escribas.

Modificar archivos de texto

EL método **.write()** se utiliza para reescribir nuevo contenido sobre el archivo de texto

```
with open("archivo.txt", "w") as archivo:  
    archivo.write("Nuevo contenido1\n")  
    archivo.write("Nuevo contenido2")
```



ACTIVIDAD

Use el método **.write()** para crear un archivo de texto que diga lo siguiente:

Mi nombre es: *su nombre*

Mis apellidos son: *sus apellidos*

Estudio en: *su institución*

El programa de: *su carrera*

Modificar archivos de texto – adicionar info

Para agregar contenido a un archivo *sin eliminar el contenido existente*, puedes abrirlo en modo adición utilizando "a".

Abrir el archivo en modo adición (append mode):

```
with open("archivo.txt", "a") as archivo:  
    archivo.write("Contenido adicional")
```

ACTIVIDAD

Lea el archivo postulación.txt

Use el método **.write()** para adicionar más líneas que digan lo siguiente:

Mi nombre es: *su nombre*

Mis apellidos son: *sus apellidos*

Estudio en: *su institución*

El programa de: *su carrera*

Vivo en: *su barrio*

Me postulo por el siguiente motivo: *su motivo*

Reemplazar una cadena en todo el archivo

```
with open("archivo.txt", "r") as archivo:
```

```
    contenido = archivo.read()
```

```
contenido_modificado = contenido.replace("Viejo_texto", "nuevo_texto")
```

```
with open("archivo.txt", "w") as archivo:
```

```
    archivo.write(contenido_modificado)
```



Reemplazar una cadena en todo el archivo

Este código abre el archivo en modo lectura, reemplaza todas las ocurrencias de 'viejo_texto' con 'nuevo_texto' en el contenido y luego sobrescribe el archivo con el contenido modificado.

Reemplazar una cadena en todo el archivo

Actividad: utilice el método `.replace()` visto anteriormente para corregir los errores ortográficos del archivo `texto_corregir.txt`



Institución
Universitaria
Reacreditada en Alta Calidad

¡MUCHAS GRACIAS!

A decorative graphic element consisting of several horizontal lines in various colors (yellow, orange, red, purple, blue, green) and a stylized white swoosh or arc on the right side.

..... Hacia una era de
Universidad y *Humanidad*